

Звіт про навантажувальне тестування RADIUS-сервера

Дата: 2026-09-18

Резюме

Проведено навантажувальне тестування RADIUS-сервера на продуктивному обладнанні. Підтверджено стабільну пропускну здатність **до ~105 000 запитів/секунду** з нульовою кількістю помилок, включно з тривалим (10-хвилинним) тестом на стабільність під рівномірним навантаженням.

Визначено та застосовано оптимальну конфігурацію сервера (розділ "Рекомендації з налаштування"), а також розраховано мінімальні технічні вимоги до обладнання для гарантованого утримання **60 000 запитів/секунду без помилок** (розділ "Технічні вимоги до обладнання").

Результати навантажувального тестування

Пропускна здатність за рівнем одночасних запитів

Тестовий стенд: сервер на 24 логічних ядрах (AMD Ryzen 9 9900X, 12 фізичних ядер, буст до 5.66 ГГц), 60 ГБ RAM, з рекомендованою конфігурацією нижче. Кожен рядок - окремий прогін тривалістю 8с.

Одночасних запитів (concurrency)	RPS (середнє)	Latency, середня	Latency, максимальна
100	74 155	1.18 мс	27 мс
200	101 836	1.83 мс	49 мс
400	105 190	3.69 мс	74 мс
800	105 374	7.46 мс	161 мс
1600	103 798	15.16 мс	273 мс

Помилки (таймаут, мережева помилка, невірна відповідь) не зафіксовано на жодному з рівнів навантаження.

Пропускна здатність швидко зростає до рівня ~200-400 одночасних запитів, далі виходить на стабільне плато ~105 000 rps. Подальше збільшення кількості одночасних запитів не підвищує пропускну здатність - лише зростає час очікування відповіді, тому оптимальний робочий діапазон для цього обладнання - **до ~400 одночасних запитів**.

Тест на стабільність (тривале навантаження)

Мета - підтвердити відсутність деградації (витоку пам'яті, падіння RPS чи зростання latency) під рівномірним навантаженням протягом тривалого часу, а не лише на короткому сплеску.

Параметри: 300 одночасних запитів, тривалість 10 хвилин, чистий старт процесу.

Показник	Значення
Тривалість	10 хв 00.001с
Запитів надіслано всього	67 512 670
Успішних відповідей	67 512 670 (100%)
Помилки	0
RPS (середнє за весь час)	112 521
Latency (мін / середня / макс)	0.03 / 2.58 / 88.84 мс
Пам'ять процесу (RSS)	стабільно ~1.3-1.5 ГБ протягом усього тесту, без тренду росту

Висновок: система стабільна під тривалим рівномірним навантаженням - за 10 хвилин і понад 67 мільйонів запитів не виявлено ні деградації пропускної здатності, ні зростання latency, ні витоку пам'яті.

Рекомендації з налаштування

Конфігурація сервера (`radius.server.conf.yml`)

Параметр	Рекомендоване значення	Обґрунтування
<code>logger.console.log_level</code>	<code>4</code>	Детальне логування (<code>6</code> , debug-рівень) призначене лише для розробки - різко знижує пропускну здатність за рахунок формування розширеного логу на кожен запит
<code>radius.read_buffer_size</code>	<code>8388608</code> (8 МБ)	Розмір буфера прийому на UDP-сокети (<code>SO_RCVBUF</code>) - запас на пікові навантаження без втрати пакетів
<code>script.pool_size</code>	<code>300</code>	Розмір пулу обробників запитів; перевірено, що подальше збільшення (до 500) не дає приросту продуктивності
<code>profiler.enabled</code>	<code>false</code>	Діагностичний інтерфейс (pprof) для продуктивного середовища вимикається; використовується лише для планової діагностики
<code>prometheus.detailed</code>	<code>false</code>	Детальна метрика з міткою по мак-адресі абонента необмежено накопичується в пам'яті процесу протягом усього часу роботи (без цього параметра пам'ять поступово зростає, у тесті - до ~6.8 ГБ за годину роботи)

Налаштування операційної системи

Розмір буфера прийому UDP-сокета, заданий у конфігурації сервера, потребує відповідного системного ліміту. Типове значення за замовчуванням в ОС (~208 КБ) недостатнє під пікові навантаження - рекомендується підняти системний ліміт заздалегідь:

```
sudo sysctl -w net.core.rmem_max=16777216
sudo sysctl -w net.core.rmem_default=8388608
```

Для збереження після перезавантаження - додати ці два параметри у `/etc/sysctl.conf` (або `/etc/sysctl.d/*.conf`).

Рекомендація щодо моніторингу

Метрика `rad_request_by_pool_count` (лічильник запитів з міткою по `pool_name`) не залежить від параметра `prometheus.detailed` і працює завжди. Кількість унікальних

значень цієї метрики обмежена реальною кількістю пулів/VLAN у мережі - для типової мережі це не є проблемою, але рекомендується періодично контролювати обсяг даних `/metrics` (`curl -s http://localhost:2112/metrics | wc -l`) при суттєвому розширенні мережі.

Технічні вимоги до обладнання

Розрахунок мінімальних вимог для гарантованого утримання **60 000 запитів/секунду без помилок**, на основі виміряного співвідношення навантаження CPU до пропускної здатності (усереднено по двох незалежних замірах: 1284% CPU на ~110 794 rps та 1414% CPU на ~101 510 rps):

Потоків CPU на 1 rps

≈ 0.000128 (середнє по замірах)

Мінімум потоків на 60 000 rps

= 60 000 × 0.000128 ≈ 7.7 → 8 потоків

Наведене значення - **теоретичний мінімум без запасу на пікові навантаження**. Для стабільної роботи в продуктивному середовищі рекомендується подвоєний запас:

Ресурс	Розрахунковий мінімум	Рекомендовано для продуктивного використання
CPU	8 потоків (≈4 фізичних ядра з технологією SMT/Hyper-Threading)	8 фізичних ядер / 16 потоків , тактова частота (турбо-буст) не нижче 4.5 ГГц
RAM	~1.5 ГБ під сам процес сервера (виміряно під час тесту на стабільність)	8 ГБ оперативної пам'яті на сервері загалом
Мережа	~192 Мбіт/с (60 000 rps × ~400 байт на обмін запит/відповідь)	Будь-який канал від 1 Гбіт/с - достатньо із суттєвим запасом

Важливе технічне зауваження: визначальним фактором продуктивності є однопотокова швидкодія процесора (тактова частота та архітектурна ефективність - IPC), а не лише кількість ядер. Обробка кожного окремого запиту виконується послідовно в одному потоці виконання (розбір протоколу, виконання логіки авторизації) і не розпаралелюється всередині одного запиту. Тому рекомендація вище прив'язана до конкретного класу процесора (сучасний десктопний/серверний процесор з високою тактовою частотою, аналогічний протестованому); менш продуктивні процесори потребуватимуть більшої кількості ядер для досягнення того самого результату.

Наведена оцінка стосується виключно самого RADIUS-процесу. Зовнішні залежності (база даних клієнтів, API-бекенд у режимі `processor: api`) до розрахунку не входять і потребують окремого масштабування відповідно до їх власного навантаження.

Методологія тестування

Навантаження створювалося спеціалізованим інструментом (`load_testing/`), що генерує Access-Request пакети, максимально наближені до реального трафіку обладнання мережі (DHCP-relay, комутатори з підтримкою option-82): випадкова MAC-адреса абонентського пристрою на кожен запит (кожен запит виглядає як нове підключення), розподіл по пулу профілів мережевого обладнання, коректно сформовані службові атрибути протоколу.

Кожен запит детерміновано отримує реальну позитивну відповідь (`Access-Accept`) від сервера - тестовий трафік не потребує попередньо підготовленої бази абонентів і не використовує жодних штучних "заглушок" на стороні сервера, тому отримані результати відображають справжню продуктивність продуктивної конфігурації.

Результат кожного прогону включає кількість запитів, частку успішних відповідей, детальну розбивку помилок за причиною (якщо такі виникають) та статистику затримки відповіді (мінімальна / середня / максимальна).

Як відтворити тестування

```
go run ./load_testing -server=<адреса>:1812 -secret=<секрет> -duration=10s -concurrency=300
```